

JavaScript: задачи на методы массива, объекты, Map/Set и async

Подборка заданий для практики и сдачи

Формат: отдельный файл для каждой задачи, запуск через node, вывод результатов в консоль

Важно: в задачах с багами объяснение ошибки нужно писать комментарием прямо над исправлением.

1. Методы массива

map · filter · reduce · find · forEach · some · every

логика · Обработка корзины интернет-магазина

Есть массив товаров. Нужно: 1) получить только товары дороже 1000₽, 2) посчитать общую сумму скидочных товаров (isOnSale: true), 3) найти первый товар категории «электроника», 4) проверить, все ли товары есть в наличии (inStock).

Каждый пункт — отдельная переменная. Не использовать циклы for/while — только методы массива.

```
const products = [
  { id: 1, name: "Наушники", price: 2500, category: "электроника", isOnSale: true, inStock: true },
  { id: 2, name: "Книга JS", price: 800, category: "книги", isOnSale: false, inStock: true },
  { id: 3, name: "Клавиатура", price: 3200, category: "электроника", isOnSale: true, inStock: false },
  { id: 4, name: "Кружка", price: 450, category: "товары для дома", isOnSale: false, inStock: true },
];
```

баг · Найди и исправь ошибки в коде

В коде ниже 3 ошибки. Найди их, объясни в комментарии почему это баг, и исправь.

```
const numbers = [1, 2, 3, 4, 5];

const doubled = numbers.map(n => { n * 2 });

const evenNumbers = numbers.filter(n => n % 2);

const sum = numbers.reduce((acc, n) => {
  return n;
}, 0);
```

логика · Статистика по студентам

Используя только методы массива, напишите функцию getStats(students), которая возвращает объект: средний балл, список отличников (≥ 90), список тех кто сдал (≥ 60), самый высокий балл.

```
const students = [
  { name: "Алия", grade: 95 },
  { name: "Марат", grade: 72 },
  { name: "Дана", grade: 58 },
  { name: "Тимур", grade: 88 },
  { name: "Зара", grade: 91 },
];
```

2. Объекты, Object.keys, копирование

Object.keys · Object.values · Object.entries · spread · structuredClone

логика · Слияние конфигов

Напишите функцию mergeConfigs(defaults, userConfig), которая возвращает итоговый конфиг. Пользовательские настройки имеют приоритет. Дефолтные настройки должны остаться нетронутыми (убедитесь, что оригинал не мутирует).

```
const defaults = { theme: "light", lang: "ru", notifications: true, fontSize: 14 };
const userConfig = { theme: "dark", fontSize: 16 };

// После вызова:
// mergeConfigs(defaults, userConfig)
// → { theme: "dark", lang: "ru", notifications: true, fontSize: 16 }
// defaults — не изменился!
```

баг · Мутация объекта — найди проблему

Разработчик хотел сделать копию юзера и изменить только копию. Но что-то пошло не так. Объясни почему, исправь, и напиши правильный вариант для вложенных объектов тоже.

```
const user = {
  name: "Алмас",
  address: { city: "Алматы", zip: "050000" }
};

const updatedUser = Object.assign({}, user);
updatedUser.name = "Бекзат";
updatedUser.address.city = "Астана";

console.log(user.name); // "Алмас" — ок
console.log(user.address.city); // ??? — сюрприз!
```

логика · Трансформация объекта через Object.entries

Напишите функцию transformPrices(prices, discount), которая принимает объект с ценами и процент скидки, возвращает новый объект с ценами со скидкой. Округляйте до целого.

```
const prices = { apple: 120, banana: 80, cherry: 350, mango: 290 };
transformPrices(prices, 10);
// → { apple: 108, banana: 72, cherry: 315, mango: 261 }
```

3. Map и Set

new Map · new Set · итерация · уникальность

логика · Подсчёт частоты слов

Используя Map, напишите функцию wordCount(text), которая возвращает Map с количеством вхождений каждого слова. Регистр не важен («Hello» и «hello» — одно

слово). Затем найдите самое частое слово.

```
const text = "the quick brown fox jumps over the lazy dog the fox";
```

логика · Дедупликация и пересечение

Используя Set: 1) удалите дубликаты из массива тегов, 2) найдите общие теги у двух статей, 3) найдите уникальные теги (которые есть только в одной из статей).

```
const article1Tags = ["js", "frontend", "react", "js", "hooks"];
const article2Tags = ["react", "typescript", "frontend", "vite"];
```

баг · Почему Map ведёт себя не так?

Разработчик использует объект как ключ в Map. Объясни, что происходит, и исправь код так чтобы он работал правильно.

```
const cache = new Map();

function fetchUser(id) {
  const key = { userId: id };
  if (cache.has(key)) {
    return cache.get(key); // никогда не попадает сюда
  }
  const result = `User #${id}`;
  cache.set(key, result);
  return result;
}
```

4. Промисы, async/await, try/catch

Promise · async/await · try/catch · Promise.all

логика · Своя функция delay и очередь запросов

1) Напишите функцию delay(ms), которая возвращает промис, разрешающийся через ms миллисекунд. 2) Используя её, напишите функцию runSequentially(tasks) — массив асинх-функций, которые должны выполняться по очереди (следующая начинается только после завершения предыдущей). Выведите время начала и конца каждой.

баг · Найди 3 ошибки в асинх-коде

В коде ниже несколько проблем. Найди их, объясни каждую и исправь.

```
async function loadUserData(userId) {
  const user = fetch(`/api/users/${userId}`);
  const posts = await fetch(`/api/posts/${userId}`);

  try {
    const data = await user.json();
    return data;
  }

  if (!user.ok) {
    throw new Error("Не удалось загрузить");
  }

  catch(err) {
    console.log("Ошибка:", err);
  }
}
```

логика · Promise.all vs последовательность

Напишите два варианта функции `loadDashboard()`: один загружает данные последовательно (пользователь → посты → комментарии), другой параллельно. Замерьте время выполнения обоих через `Date.now()`. Объясните в комментарии когда что использовать.

```
const getUser = () => new Promise(res => setTimeout(() => res({ name: "Алия" }), 500));
const getPosts = () => new Promise(res => setTimeout(() => res([1,2,3]), 700));
const getComments = () => new Promise(res => setTimeout(() => res(["ок", "круто"]), 400));
```

сложная • Повторная попытка при ошибке (retry)

Напишите функцию `withRetry(fn, retries, delayMs)`, которая вызывает асинх-функцию `fn`. Если она выбрасывает ошибку — повторяет попытку через `delayMs` миллисекунд, но не более `retries` раз. Если все попытки провалились — выбрасывает последнюю ошибку.

```
let attempts = 0;
const unreliable = async () => {
  attempts++;
  if (attempts < 3) throw new Error(`Попытка ${attempts} провалилась`);
  return "Успех!";
};

// withRetry(unreliable, 5, 200) → "Успех!" (с 3-й попытки)
```

5. Мини-проект — «Менеджер задач»

Объединяет все темы

CLI-подобный менеджер задач

Напишите класс TaskManager (или набор функций), который умеет управлять задачами. Требования ниже — реализуйте их все.

Что должно уметь приложение:

1. `addTask(title, tags[])` — добавить задачу (id через Map, теги через Set)
2. `completeTask(id)` — отметить как выполненную
3. `deleteTask(id)` — удалить задачу
4. `getByTag(tag)` — получить все задачи с нужным тегом
5. `getStats()` — { total, completed, pending, allTags: Set }
6. `saveToStorage()` — async функция, симулирует сохранение (delay 300ms)
при неудаче — бросает ошибку, вызывающий код должен её поймать
7. `loadFromStorage()` — async, симулирует загрузку

Условия:

- Используйте Map для хранения задач (ключ — id)
- Используйте Set для тегов каждой задачи
- Все async операции — с try/catch
- Используйте методы массива для фильтрации/поиска

Это основное задание. Если сделали всё выше — уделите максимум времени именно ему.

Правила сдачи

Писать код руками — не копировать, не генерировать AI

Каждый файл — отдельная задача (task1.js, task2.js...)

В задачах с багами — объяснить ошибку в комментарии над исправлением

Проект должен запускаться через node и выводить результаты в консоль

Можно задавать вопросы в чате группы, но код пишем сами